LOT COMPUTER ��? API-CONNECTOR

LOT COMPUTER — LOT API Connector Specification

```
DOCUMENT   LOT-COMPUTER / 04 API CONNECTOR   REV A   2026-10-11
S-2        VADIK MARMELADOV
STATUS     SPEC — server endpoints below do NOT exist yet
```

1. What exists today (read from repo)

- Fastify routes in `src/server/routes/api.ts`, mounted under `/api`.
- `POST /api/logs` body `{ text, event?, metadata? }` creates a Log row for the authenticated `req.user` (event defaults to `note`; metadata is free JSON).
- `GET /api/logs`, `GET /api/weather`, `GET /api/sync` (SSE) exist.
- Auth is **user-session based** (`req.user`). Devices have no session, so the connector needs a **device token** layer (new).

2. New: device identity

Table `devices(id, user_id, serial, name, token_hash, fw_version, last_seen, created_at)` via a new migration in `migrations/`. Token: random 256-bit, shown once at pairing, stored hashed. Header `Authorization: Bearer <token>`. Device requests resolve to the owning user, so existing `Log.create` logic is reused. Rate-limit per device; revoke from Settings.

3. Endpoints (proposed)

Method & path	Purpose
`POST /api/device/pair`	exchange one-time pairing code (shown in site Settings) for token
`GET /api/device/notifications?since=<id>`	pager feed: `[{id, text, kind, ts, expires}]`
`POST /api/device/digest`	session digest (CBOR or JSON), stored in Log metadata
`POST /api/device/ack`	notification seen/dismissed
`GET /api/device/config`	poll interval, quiet hours, feature flags
`POST /api/device/photo`	optional JPEG (Q-1), size-capped

Copy button reuses the **existing** `POST /api/logs`, device-authenticated:

```
        { "text": "Coffee time!", "event": "device_copy",
          "metadata": { "device": "LOT-C-0001", "notification": 4821, "fw": "0.1.0", "battery": 71 } }
```

so it appears in the Log tab with no new UI beyond rendering `device_copy`.

4. Notification generation (“AI-powered”)

A server job (like the existing scheduled QIE jobs) uses the user’s context (time, weather, plans, memory-engine summaries) to produce short notifications (≤ 64 chars, plain text, no emoji per LOT style) and writes them to a `device_notifications` table. Notifications are a *proposal*: user can mute kinds and set quiet hours on the site. Every generated notification is itself logged.

5. Session digest

See firmware §5. Server stores the digest as Log `event: "device_session"` with metadata; counts toward the memory-engine compression pipeline.

6. Security

TLS only, token hashing, per-device rate limits, payload size caps, no PII in notification text beyond what the user wrote, photo upload only if enabled, audit row on pair/revoke. Device clock skew tolerated; server timestamps win.

7. Implementation plan (P1b)

1. migration `devices`, `device_notifications` 2. auth hook for `/api/device/*`
2. endpoints + tests 4. Settings UI (pair, revoke, mute) 5. Log tab renderer
3. staging soak with the dev kit. Not implemented this session: the repo's build tooling was not installed in this environment, and unverified server code must not ship.